NetPulse IoT Platform

Comprehensive Technical Documentation, ML Architecture & Q&A; Manual

Project: NetPulse IoT Security Platform	**Classification:** Enterprise IoT Defense / SIEM
Backend: FastAPI (Python 3.13) + SQLite/PostgreSQL	**Frontend:** React 19 + TypeScript + Tailwind v4
ML Engine: Scikit-Learn Random Forest Classifier	**Hardware Agent:** Physical ESP32 (C++ Arduino Core)

1. Executive Summary & Purpose

NetPulse IoT is an enterprise-grade IoT network security monitoring, anomaly detection, and automated threat mitigation platform. Modern Internet of Things (IoT) ecosystems—such as smart sensors, surveillance cameras, industrial edge nodes, and smart home appliances—frequently suffer from hardcoded credentials, unpatched firmware, and lack of onboard security agents. NetPulse IoT provides continuous zero-trust traffic surveillance, dynamic multi-factor risk scoring (0–100), Random Forest threat classification, and automated quarantine containment.

- **Real-Time Telemetry & Flow Ingestion:** Continuously ingests volumetric metrics, request frequencies, packet rates, and handshake telemetry from physical microcontrollers (ESP32) and heterogeneous IoT device fleets.
- **AI-Powered Anomaly Classification:** Employs an ensemble Random Forest Machine Learning classifier to differentiate normal operational baseline telemetry from multi-vector malicious cyberattacks.
- **Automated Zero-Touch Quarantine:** Executes real-time mitigation policies, automatically isolating high-risk nodes (temporary TTL quarantine or permanent block) to prevent lateral botnet spread without requiring human intervention.
- **Transparent Risk & Domain Intelligence:** Calculates explainable 0–100 risk scores with natural-language rationale and provides live domain heuristic telemetry mapping external destination risks.

2. Target Users & Customer Landscape

Role / Stakeholder	Operational Use Case & Value Proposition
SOC Analysts & Security Engineers	Monitors live threat feeds, investigates high-risk alerts, inspects contacted external domains, reviews explainable ML decision trees, and issues manual quarantine overrides.
IoT & Network Administrators	Maintains inventory of active physical (ESP32) and virtual IoT assets, audits network health, monitors bandwidth consumption, and configures auto-blocking thresholds.
Industrial & SCADA Engineers	Protects vulnerable legacy operational technology (OT) and sensor nodes in manufacturing, oil & gas, and utilities from unauthorized remote command-and-control (C2) intrusion.
Enterprise IT & Healthcare Staff	Ensures HIPAA / SOC2 compliance across hospital smart medical equipment, corporate smart office sensors, access control systems, and CCTV camera arrays.

3. Comprehensive Technology Stack

Layer	Technologies Used	Key Architectural Function
Backend Core	Python 3.13, FastAPI, Uvicorn, Pydantic v2	Asynchronous RESTful APIs, device event endpoints, background worker loops, and high-throughput routing.
Database Layer	SQLite (Local Default) / PostgreSQL, SQLAlchemy ORM	12 Relational tables tracking devices, telemetry events, alerts, domains, blocked entities, audit logs, and risk records.
Machine Learning	Scikit-Learn, Pandas, NumPy, Joblib	RandomForestClassifier for flow classification, automated self-training fallback, feature importance extraction.
Real-Time Feed	Native WebSockets (FastAPI WebSocket Manager)	Bidirectional broadcast streaming live telemetry, new alerts, and quarantine updates to connected UI clients.
Frontend SaaS	React 19, TypeScript, Vite, Tailwind CSS v4, Recharts, Lucide	Enterprise dashboard with live graphs, SVG Domain Risk Radar, quarantine countdown clocks, and CSV export.
IoT Firmware	ESP32 C++ (Arduino Framework), WiFiMulti, HTTPClient	Standalone agent with Wi-Fi watchdog, REST JSON telemetry dispatch, physical DHT sensor support & LED diagnostics.
Security & Auth	PyJWT (JSON Web Tokens), Passlib Bcrypt	Role-based access control (Admin/Analyst), token expiration, and secure password hashing.

4. Telemetry Dataset & Attack Modeling

The model is trained on a rigorously structured IoT flow telemetry dataset comprising **12,000 balanced flow samples**. The dataset models realistic operational baselines (60%) alongside labeled multi-vector cyberattack patterns (40%) with realistic stochastic network jitter.

Extracted Feature (9 Total)	Type / Unit	Diagnostic Significance
traffic_volume	Integer (Bytes)	Detects volumetric flooding, data exfiltration, or DDoS floods.
packet_count	Integer (Packets)	Monitors burst intensity and packet fragmentation anomalies.
request_frequency	Float (Req / sec)	Identifies aggressive polling or API endpoint abuse.
connection_frequency	Float (Conn / min)	Highlights rapid port scanning and brute-force sweeps.
failed_connections	Integer (Count)	Captures failed TCP SYN handshakes and rejected authentication.
protocol_num	Integer (6=TCP, 17=UDP)	Distinguishes connection-oriented vs UDP broadcast streams.
destination_port	Integer (Port #)	Identifies communication over vulnerable or malicious ports.
event_rate	Float (Events / min)	Measures telemetry dispatch velocity from edge devices.
historical_alerts_count	Integer (Count)	Factors past suspicious behavior into current classification.

Simulated Cyberattack Profiles in Dataset:

- **Volumetric DoS/DDoS Flood:** Massive byte surges (18k–180k bytes), elevated packet bursts (220–2200 packets), high connection frequency (40–260 conn/min).
- **Port Scan & Reconnaissance:** Widespread failed connections (12–75 failures) directed across SSH (22), Telnet (23), SMB (445), RDP (3389), and HTTP Alt (8080).
- **Brute-Force Authentication:** Repeated connection velocity targeted at authentication endpoints (ports 22, 23, 554 RTSP) with elevated handshake failure rates.
- **Command & Control (C2) Beaconing:** Periodic bursts to unusual high ports (4444, 6667, 9001, 1337) representing botnet control channel handshakes.
- **Abnormal High-Frequency Polling:** Excessive request frequencies (5.5–30 req/sec) exceeding standard IoT device telemetry limits.

5. Machine Learning Algorithm & Selection Rationale

Selected Algorithm: RandomForestClassifier (Scikit-Learn Ensemble).
Hyperparameters: 100 Estimators (Trees), Max Depth = 12, Min Samples Split = 5, Min Samples Leaf = 2, Class Weight = 'balanced'.

Evaluation Criterion	Random Forest Advantage in NetPulse IoT	Alternative Comparison
Non-Linear Decision Boundaries	Captures complex multi-feature interactions (e.g. high volume + unusual port + failed handshakes).	Logistic Regression fails on complex non-linear boundary intersections.
Overfitting Resilience & Bagging	Ensemble aggregation of 100 decision trees averages out random IoT network jitter and packet spikes.	Single Decision Trees overfit to noise; k-NN degrades in high dimensional spaces.
Explainability & Transparency	Provides exact feature importance metrics, allowing the system to output clear natural-language rationale for SOC operators.	Deep Neural Networks / Multi-Layer Perceptrons act as unexplainable 'black boxes'.
Inference Latency	Sub-millisecond CPU inference time (< 1.5ms per flow), ensuring zero bottlenecks on live streaming traffic.	Heavy Deep Learning models require GPUs and introduce excessive stream latency.
Class Imbalance Handling	Native class_weight='balanced' balances minority attack vectors against dominant normal traffic.	Standard Naive Bayes can be heavily skewed by class frequency differentials.

6. Model Training Workflow & Automated Self-Healing

Are we training a model? YES.
Why are we training a model? Rule-based firewalls and static signature matching fail against unknown or evolving attack vectors. Training an ML classifier allows the system to learn multivariate statistical correlations across flow metrics. The model is trained on a 75/25 stratified train-test split, yielding outstanding validation metrics:

Model Accuracy: > 99.4%	**Precision:** > 99.2%	**Recall:** > 99.5%	**F1 Score:** > 99.3%

Automated Self-Healing Boot Mechanism: The backend incorporates a resilient startup hook. If the compiled model artifact (rf_iot_model.joblib) is not found upon starting the FastAPI application, the backend automatically generates the 12,000-sample training dataset, trains the Random Forest classifier in memory, exports the model and metadata JSON, and begins live inference without requiring manual developer intervention.

7. Redirection, Mitigation & Automated Quarantine

NetPulse IoT implements an active mitigation pipeline driven by its **Multi-Factor 0–100 Risk Scoring Engine**. When an incoming telemetry event is analyzed, the composite risk score dictates the automated security policy:

Risk Score	Risk Tier	Automated Action Taken	Remediation / Redirection Flow
81 – 100	**Critical**	**Automated Permanent Block**	Device is immediately quarantined in blocked_entities. All incoming packets are rejected (HTTP 403 Forbidden). Critical alert broadcast over WebSocket.

61 – 80	**High**	**Automated Temporary Quarantine**	Entity is isolated for **15 minutes (configurable TTL)**. Live countdown clock displays on frontend. Background worker restores normal status once timer expires.
31 – 60	**Medium**	**Alert Generated & Monitored**	Device state switched to suspicious. Traffic logged with high-resolution audit flags. Operator notified on live alert feed.
0 – 30	**Low**	**Normal Baseline Operation**	Traffic accepted and indexed in telemetry history. Normal operational state maintained.

Asynchronous Quarantine Expiration Worker: A dedicated background task runs continuously every **5 seconds** inside the FastAPI engine. It queries the database for expired temporary blocks, deactivates the block record, restores device status to online, and broadcasts a WebSocket notification to synchronize all connected dashboards in real time.

8. Database Architecture: Is it Local?

Which Database is used? The system uses a relational database managed through **SQLAlchemy ORM**.
Is it Local? YES. By default, NetPulse IoT operates on a high-performance local **SQLite database file (netpulse.db)** stored directly in the project root directory. This ensures 100% zero-configuration, standalone portability on any laptop or offline edge gateway.
Enterprise Production Ready: The database connection string is decoupled via the DATABASE_URL environment variable. Switching to production-grade PostgreSQL or Cloud SQL requires only a single environment variable change with zero code modifications.

Table Name	Data Stored & Entity Role
users	User credentials (hashed bcrypt), names, roles (Admin/Analyst), and active flags.
devices	Hardware inventory: device ID, IP/MAC address, online/blocked status, risk score, sensor type.
device_events	Time-series flow telemetry: volume, packet count, frequencies, ports, risk predictions, explanations.
network_traffic	Aggregate throughput telemetry (bytes in/out, packets) used for real-time charting.
alerts	Security incidents: alert ID, severity, description, status (New/Resolved), and mitigation action.
domains & domain_activities	Contacted external domain records, overall risk ratings, categories, and per-device query activities.
blocked_entities & security_actions	Active & expired quarantine blocks, TTL timestamps, and comprehensive audit trail of security decisions.

risk_score_records & simulation_sessions	Historical composite score logs and active fleet simulation status configurations.

9. Physical ESP32 Microcontroller Integration

NetPulse IoT includes genuine, compilable C++ firmware designed for physical **ESP32 Microcontrollers** (Dev Module, NodeMCU-32S, ESP32-WROOM-32). The ESP32 acts as a real hardware edge client streaming environmental and network telemetry into the backend.

- **Multi-Wi-Fi Auto-Discovery:** Built using WiFiMulti to seamlessly connect between mobile hotspots and home/office Wi-Fi without code re-flashing.
- **Dual Deployment Modes:** Supports instant switching between Local Mode (LAN/Hotspot laptop IP) and Online Cloud Mode (HTTPS ngrok/Render/Railway) via a single preprocessor directive (USE_ONLINE_BACKEND).
- **Intelligent LED Diagnostics:** Provides clear hardware feedback via GPIO 2: Rapid blinking on Wi-Fi connection, double-blink on transmission, and solid indicator on error.
- **Physical & Fallback Telemetry:** Reads real environmental data from DHT11/DHT22 sensors wired to GPIO 4, with automatic realistic sensor simulation fallback if no physical sensor is attached.
- **Self-Registering Edge Agent:** Automatically registers into the NetPulse dashboard upon first transmission with source tag Real.

10. Master Project Q&A; Matrix

Q1: What exact technologies are used across the full NetPulse IoT stack?

NetPulse IoT utilizes Python 3.13 and FastAPI for the high-concurrency asynchronous backend; SQLAlchemy ORM with SQLite (local default) and PostgreSQL support for data persistence; Scikit-Learn (Random Forest Classifier) with Pandas/NumPy for machine learning; React 19, TypeScript, Vite, Tailwind CSS v4, and Recharts for the modern frontend dashboard; Native WebSockets for real-time live event streaming; and C++ Arduino Core with WiFiMulti and HTTPClient for physical ESP32 microcontrollers.

Q2: Who is the user of this platform?

The primary users are Security Operations Center (SOC) Analysts, IoT / Network Administrators, and Industrial Systems Security Officers who need continuous visibility, real-time threat detection, and automated mitigation capabilities across enterprise and industrial IoT hardware fleets.

Q3: For what purpose is NetPulse IoT being used?

NetPulse IoT is used to monitor, detect, and automatically quarantine anomalous or malicious behavior in IoT networks (such as DDoS flooding, brute-force intrusions, port scanning, and command-and-control communication). It prevents compromised smart devices from pivoting attacks across critical local and enterprise networks.

Q4: Who are the target customers and business beneficiaries?

Target customers include Smart Manufacturing & Industry 4.0 facilities, Healthcare Hospitals (protecting connected medical devices), Smart City & Building Operators, Corporate Enterprise Offices (securing CCTV and smart appliances), and Managed Security Service Providers (MSSPs) delivering IoT security-as-a-service.

Q5: What dataset is used, and how was it constructed?

The dataset comprises 12,000 balanced flow telemetry samples capturing 9 extracted features (traffic volume, packet count, request frequency, connection velocity, failed handshakes, protocol, port, event rate, and historical alerts). It models 60% normal device baseline operations and 40% labeled multi-vector cyberattacks (DDoS floods, port reconnaissance, brute force, C2 beacons, abnormal polling).

Q6: Which machine learning algorithm is used, and why was it chosen?

The Random Forest Classifier (from Scikit-Learn) was chosen. Key reasons include: (1) Superior capture of non-linear multi-feature interactions, (2) High resilience against noise and packet jitter via ensemble tree bagging, (3) Exact feature importance interpretability for natural-language alert explanations, (4) Sub-millisecond inference speeds suitable for live streaming, and (5) Built-in class weight balancing.

Q7: Are you training a model? Why are you training a model?

Yes, we train a model. We train a model because static rule-based firewalls cannot detect complex, multivariate, or evolving attack vectors. Machine Learning identifies subtle mathematical anomalies across multidimensional flow features. NetPulse also includes an automated self-healing fallback that automatically generates data and trains the model on first boot if the artifact is missing.

Q8: How does redirection, mitigation, and automated quarantine work?

The system evaluates a dynamic 0–100 composite risk score. Scores >= 81 trigger an immediate Permanent Block; scores 61–80 trigger an Automated Temporary Block (quarantine) with a 15-minute countdown timer; scores 31–60 trigger suspicious monitoring. Quarantined devices have incoming traffic rejected with HTTP 403 Forbidden. A background worker running every 5 seconds automatically expires temporary blocks and restores devices to online status.

Q9: Which database is used? Is it local?

NetPulse IoT uses a relational database via SQLAlchemy ORM. By default, it runs on a local SQLite database (netpulse.db) requiring zero external setup. It is fully production-ready and can connect to external PostgreSQL or Cloud SQL databases simply by setting the DATABASE_URL environment variable.

Q10: What makes the physical ESP32 integration unique?

The ESP32 is a genuine physical hardware client with standalone power bank capability, multi-SSID auto-discovery, dual local/cloud deployment modes, hardware LED diagnostics, DHT11/22 physical sensor telemetry, and automatic registration into the live React dashboard.